

conference1

编辑距离讲义

一、定义

编辑距离是指两个字符串之间，由一个转换成另一个所需的最少编辑操作次数。

允许的编辑操作：

操作	说明	示例
插入	在任意位置插入一个字符	"ab" → "acb"
删除	删除任意一个字符	"abc" → "ac"
替换	将一个字符替换为另一个	"abc" → "adc"

每进行一次上述操作，距离增加1。编辑距离越小，字符串越相似。

二、动态规划思想

2.1 核心原理

编辑距离具有最优子结构性质：计算两个字符串的编辑距离时，可以分解为计算它们前缀之间的编辑距离。通过自底向上地填充一个二维表格，最终得到完整字符串的距离。

2.2 状态定义

设两个字符串分别为 A 和 B，长度分别为 m 和 n。

定义 $dp[i][j]$ 表示 A 的前 i 个字符构成的子串与 B 的前 j 个字符构成的子串之间的编辑距离。

2.3 状态转移方程

边界条件：

- $dp[0][j] = j$ ，当 A 为空串时，需要插入 j 次
- $dp[i][0] = i$ ，当 B 为空串时，需要删除 i 次

当 $i > 0$ 且 $j > 0$ 时:

- 若 $A[i-1] == B[j-1]$, 则 $dp[i][j] = dp[i-1][j-1]$
- 若 $A[i-1] != B[j-1]$, 则:

```
dp[i][j] = 1 + min(  
    dp[i-1][j],      // 删除 A[i-1]  
    dp[i][j-1],      // 插入 B[j-1]  
    dp[i-1][j-1]     // 替换 A[i-1] 为 B[j-1]  
)
```

三、三种计算方式

3.1 递归（自顶向下）

直接按照定义编写递归函数，效率低（指数级复杂度），存在大量重复计算。

```
function edit_distance(A, i, B, j):  
    if i == 0: return j  
    if j == 0: return i  
    if A[i-1] == B[j-1]:  
        return edit_distance(A, i-1, B, j-1)  
    else:  
        return 1 + min(  
            edit_distance(A, i-1, B, j),      // 删除  
            edit_distance(A, i, B, j-1),      // 插入  
            edit_distance(A, i-1, B, j-1)     // 替换  
        )
```

3.2 带备忘录的递归（自顶向下 + 记忆化）

在递归基础上增加一个二维数组记录已计算过的子问题，避免重复计算。时间复杂度降为 $O(m \times n)$ 。

初始化 memo 为 -1

```
function edit_distance_memo(A, i, B, j):  
    if i == 0: return j  
    if j == 0: return i  
    if memo[i][j] != -1: return memo[i][j]  
  
    if A[i-1] == B[j-1]:
```

```
        memo[i][j] = edit_distance_memo(A, i-1, B, j-1)
    else:
        memo[i][j] = 1 + min(
            edit_distance_memo(A, i-1, B, j),
            edit_distance_memo(A, i, B, j-1),
            edit_distance_memo(A, i-1, B, j-1)
        )
    return memo[i][j]
```

3.3 迭代（自底向上）

使用二维数组按行或按列填充，是实际最常用的方法。

```
function edit_distance_iter(A, B):
    m = len(A), n = len(B)
    dp = 二维数组 (m+1) × (n+1)

    // 初始化边界
    for i = 0 to m:
        dp[i][0] = i
    for j = 0 to n:
        dp[0][j] = j

    // 填充表格
    for i = 1 to m:
        for j = 1 to n:
            if A[i-1] == B[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(
                    dp[i-1][j],    // 删除
                    dp[i][j-1],    // 插入
                    dp[i-1][j-1]   // 替换
                )
    return dp[m][n]
```

空间优化：由于每一行只依赖上一行，可以将空间复杂度从 $O(m \times n)$ 降至 $O(\min(m, n))$ 。

四、编辑距离自动机

4.1 定义

编辑距离自动机是一种有限状态自动机，用于判断一个字符串与给定模式串的编辑距离是否不超过某个阈值 k 。

4.2 构造原理

对于模式串 P 和阈值 k ，可以构造一个自动机，其状态表示当前已处理前缀与 P 的编辑距离。自动机逐字符读入文本串，状态转移时更新编辑距离。

4.3 状态表示

状态可以表示为长度为 $|P|+1$ 的向量，其中第 i 个元素表示当前文本前缀与 P 的前 i 个字符的最小编辑距离。

当 k 较小时，可以使用**裁剪技术**，只保留距离 $\leq k$ 的状态，大幅减少状态数量。

4.4 应用场景

- 在线拼写检查
- 模糊字符串匹配
- DNA序列近似匹配

五、自顶向下与自底向上对比

维度	自顶向下（递归+备忘录）	自底向上（迭代）
计算顺序	从原问题出发，按需递归	从小规模子问题开始，逐步扩大
表格填充	只计算实际需要的状态	计算所有状态
实现难度	较简单，逻辑清晰	需要设计遍历顺序
空间利用率	依赖递归栈	可以滚动数组优化
适用场景	状态转移不规则时	状态空间完整时

六、计算示例

计算 $A = \text{"cat"}$ 与 $B = \text{"car"}$ 的编辑距离。

手动推导：

1. 比较 c vs c ：相同，距离 = 0

- 2. 比较 `a` vs `a`：相同，距离 = 0
- 3. 比较 `t` vs `r`：不同，需要一次替换

最终距离 = 1

动态规划表格 (`dp[i][j]`):

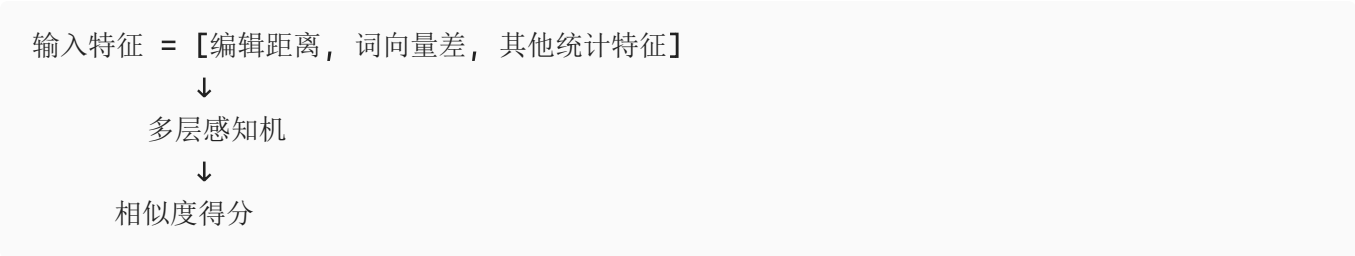
	∅	c	a	r
∅	0	1	2	3
c	1	0	1	2
a	2	1	0	1
t	3	2	1	1

右下角 `dp[3][3] = 1`，即编辑距离为 1。

七、在多层感知机中的具体应用

7.1 文本相似度特征

在多层感知机处理文本对任务（如文本匹配、重复问题检测）时，可以将编辑距离作为一个特征输入：



7.2 序列生成中的损失辅助

在序列生成任务（如机器翻译、语音识别）中，可以将编辑距离作为辅助损失项，与交叉熵损失联合优化：

总损失 = $\alpha \times$ 交叉熵损失 + $\beta \times$ 编辑距离损失

7.3 数据增强

使用编辑距离生成训练样本的变体：

- 对原始样本随机执行插入、删除、替换操作
- 生成与原样本编辑距离为 1~3 的扰动样本
- 增强模型对输入噪声的鲁棒性

7.4 代码示例

```
import numpy as np

def edit_distance(a, b):
    m, n = len(a), len(b)
    dp = np.zeros((m+1, n+1))
    for i in range(m+1):
        dp[i][0] = i
    for j in range(n+1):
        dp[0][j] = j
    for i in range(1, m+1):
        for j in range(1, n+1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]

# 在 MLP 中使用编辑距离作为特征
def prepare_features(s1, s2):
    dist = edit_distance(s1, s2)
    normalized_dist = dist / max(len(s1), len(s2))
    return np.array([normalized_dist])
```

7.5 注意事项

- **归一化**：原始编辑距离受字符串长度影响，建议除以最大长度进行归一化
- **阈值截断**：当距离超过一定阈值时，区分能力下降，可截断处理
- **与其他特征结合**：编辑距离单独使用效果有限，常与语义特征（如BERT嵌入）结合